Accessing object representations

Document #:	P1839R7
Date:	2025-01-11
Project:	Programming Language C++
Audience:	CWG

Reply-to: Timur Doumler <papers@timur.audio>
Krystian Stasiowski <sdkrystian@gmail.com>
Brian Bi <bbi10@bloomberg.net>

Contents

§ 1 Abstract

This paper proposes a wording fix to the C++ standard to allow read access to the object representation (i.e. the underlying bytes) of an object. This is valid in C, and is widely used and assumed to be valid in C++ as well. However, in C++ this is is undefined behaviour under the current specification.

§ 2 Motivation

Consider the following program, which takes an `int` and prints the underlying bytes of its value in hex format:

```
void print_hex(int n) {
  unsigned char* a = (unsigned char*)(&n);
  for (int i = 0; i < sizeof(int); ++i)
    printf("%02x ", a[i]);
}

int main() {
  print_hex(123456);
}
```

In C, this is a valid program. On a little-endian machine where `sizeof(int) == 4`, this will print `40 e2 01 00`. In C++, this is widely assumed to be valid as well, and this functionality is widely used in existing code bases (think of binary file formats, hex viewers, and many other low-level use cases).

However, surprisingly, in C++ this code has undefined behaviour under the current specification. In fact, it is impossible in C++ to directly access the object representation of an object (i.e. to read its underlying bytes), even for built-in types such as `int`. Instead, we would have to use `memcpy` to copy the bytes into a separate array of `unsigned char`, and access them from there.[1] However, this workaround only works for trivially copyable types. It also directly violates one of the fundamental principles of C++: to leave no room for a lower-level language.

The goal of this paper is to provide the necessary wording fixes to make accessing object representations such as in the code above defined behaviour. Existing compilers already assume that this should be valid. The goal of the paper is therefore to *not* require any changes to existing compilers or existing code, but to legalise existing code that already works in practice and was always intended to be valid.

§ 3 The problem

The cast to `unsigned char*`, which performs a `reinterpret_cast`, is fine, because `char`, `unsigned char`, and `std::byte` can alias any other type, so we do not violate the rules for type punning. However, with the current wording, this cast does *not* yield a pointer to the first element of `n`'s object representation (i.e. a pointer to a byte), and in fact it is currently impossible in C++ to obtain such a pointer. This is because this particular `reinterpret_cast` is exactly equivalent to `static_cast<unsigned char*>(static_cast<void*>(&n))` as per §7.6.1.10 [expr.reinterpret.cast][2]p7, and as such, §7.6.1.9 [expr.static.cast]p13 dictates that the value of the pointer is unchanged and therefore it points to the original object (the `int`). When `a` is dereferenced, the behaviour is undefined as per §7.1 [expr.pre]p4 because the value of the resulting expression would *not* be the value of the first byte, but the value of the whole `int` object (123456), which is not a value representable by `unsigned char`.

Further, even if we ignore this issue, `a` does not point to an array of `unsigned char`, because such an array has never been created, and therefore pointer arithmetic on `a` has undefined behaviour. An object representation as defined by §6.8 [basic.types]p4 is merely a *sequence* of `unsigned char` objects, not an array, and is therefore unsuitable for pointer arithmetic. No array is ever created explicitly, and no operation is being called in the above code that would implicitly create an array, since casts are not operations that implicitly create objects as per §6.7.2 [intro.object]p11.

It is possible to explicitly start the lifetime of an array of `unsigned char` in the storage occupied by `n` whose values are the values of `n`'s object representation. This can be done by using `std::memmove` to copy `n` to itself or, since C++23, calling the `std::start_lifetime_as_array` function. However, these operations are destructive: because the new array reuses the storage of `n`, `n`'s lifetime ends when the new array comes into existence. In a multithreaded program, this operation can race with another operation that reads `n`, and is therefore less useful than copying the bytes into a separate array in order to examine them.

§ 4 History and context

The intent of CWG has always been that the above code should work, as exemplified by [CWG1314], in which it is stated that access to the object representation is intended to be well-defined. Further, it seems that the above code actually *did* work until C++17, when [P0137R1] was accepted. This proposal fixed an unrelated core issue and included a change to how pointers work, notably that they point to objects, rather than just representing an address. It seems that the proposal neglected to add any provisions to allow access to the object representation of an object, and thus inadvertently broke this functionality. Therefore, this paper is a defect report, not a proposal of a new feature.

Notably, there are even standard library facilities that directly use this functionality and cannot be implemented in standard C++ without fixing it. One such facility is `std::as_bytes` (introduced in C++20), which obtains a `std::span<const std::byte>` view to the object representation of the elements of another span. Now, we do have a few "magic" functions in the C++ standard library that cannot be implemented in standard C++, but reading the underlying bytes of an object is such basic functionality that it should not fall into this category.

§ 5 Non-goals

This paper does not propose to make in-place modification of the object representation valid, i.e. *writing* into the underlying bytes, only *reading* them. The following code will still have undefined behaviour:

```
void increment_first_byte(int* n) {
  auto* a = reinterpret_cast<char*>(n);
  ++(*a);
}
```

It may be desirable to allow such code as well. However, unlike reading the object representation, the effect of modifying it has never been specified in C++, so specifying it would be a new feature, not a defect report. Therefore, CWG gave the guidance to reduce the scope of this paper to reading only, and propose the modifying case in a separate paper (not yet published).

This paper also does not propose to subvert existing type punning rules in any way. The proposed changes will not allow type punning between two different types where it was not previously allowed, such as between `int` and `float` (this should be done using `std::bit_cast`). It only allows type punning to `char`, `unsigned char`, and `std::byte`, which are already allowed to alias any other type.

We also do not propose to make accessing the object representation work for *all* types in C++, only for types that are currently guaranteed to occupy contiguous bytes of storage, that is, for trivially copyable or standard-layout types as per §6.7.2 [intro.object]p8. On the one hand, this is unnecessarily restrictive: in practice, any sane implementation will have complete objects, array elements, and member subobjects occupying contiguous memory, as the only reason an object would need to be non-contiguous would be if it was a virtual base subobject. On the other hand, making more objects contiguous (and therefore, their object representations accessible) is not in scope for this paper, and is instead tackled in a separate proposal [P1945R0].

§ 6 Proposed solution

For an object *a* of type `T`, we propose to change the definition of *object representation* to be considered an array of `unsigned char`, and not merely a *sequence* of `unsigned char` objects, if `T` is a type that occupies contiguous bytes of storage. We propose that this object representation should be an object in its own right, occupying the same storage as *a* and having the same lifetime. This will make pointer arithmetic work with a pointer to an element of the object representation.

To avoid an infinite recursion of nested object representations, we further specify that an array of `unsigned char` acts as its own object representation. We also need to prevent implicit object creation [P0593R6] within object representations.

We further propose that obtaining a pointer to the object representation should be possible through the use of a cast to `char`, `unsigned char`, or `std::byte`, and allow this pointer to be cast back to a pointer to its respective object. For this, we need to make the appropriate changes to the specification of `static_cast` and to make *a* pointer-interconvertible with its own object representation as well as with the first element thereof. We need to do this in a way that preserves `reinterpret_cast`'s equivalence with `static_cast` with respect to converting object pointers. Simultaneously, if multiple pointer-interconvertible objects exist, we need to specify which one is chosen.

Additionally, we need to make reading an object representation through a pointer to `char` or `std::byte` well-defined, even though it points to an element of the object

representation which is of type `unsigned char`. In these cases, we must allow for the type of the expression to differ from that of the object pointed to.

We also need to say something about the values of the elements of an object representation. We propose that for objects of type `char`, `unsigned char`, and `std::byte`, the value of each element is the value of the object it represents. For all other types, the values of the elements of the object representation are unspecified. It seems extremely difficult to specify for the general case what the value of each element would be, but it is also unnecessary, since our goal is only to make reading the elements well-defined, not to specify a particular result (which won't be the same across platforms).

Finally, multiple objects may occupy the same storage, in which case the objects' respective object representations will overlap. We must therefore adjust the specification of `std::launder` to define which object it will return a pointer to.

In order to preserve reachability-based restrictions that currently exist in C++, we propose that object representations of subobjects are distinct arrays that are simply allowed to overlap in memory with object representations of their enclosing objects. Therefore, a pointer to an element of an object representation that is obtained by a `reinterpret_cast` applied to a pointer to *a1* cannot be used to "escape" from the bytes of *a1* and reach bytes of *a2* that exist outside *a1*.[3]

§ 7 Polls

EWGI

Should accessing the object representation be defined behavior?

> Unanimous consent

Forward P1839R1 as presented to EWG, recommending that this be a core issue?

> Unanimous consent

EWG

It should be possible to access the entire object representation through a pointer to a char-like type as a DR.

SF	F	N	A	SA
10	8	2	0	0

> Consensus

§ 8 Proposed wording

The reported issue is intended as a defect report with the proposed resolution as follows. The effect of the wording changes should be applied in implementations of all previous versions of C++ where they apply. The proposed changes are relative to the C++ working draft [N5001].

Modify §6.7.2 [intro.object]p3 as follows:

> If a complete object is created ([expr.new]) in storage associated with another object *e* of type "array of *N* `unsigned char`" other than a synthesized object representation ([basic.types.general]) or of type "array of *N* `std::byte`" ([cstddef.syn]), that array *provides storage* for the created object if […]

Modify §6.7.2 [intro.object]p4 as follows:

> An object *a* is *nested within* another object *b* if
>
> — *a* is a subobject of *b*, or
>
> — *b* provides storage for *a*, or
>
> — there exists an object *c* where *a* is nested within *c*, and *c* is nested within *b*.
>
> [*Note*: An object representation is not nested within any other object representation. —*end note*]

Modify §6.7.2 [intro.object]p10 as follows:

> Unless an object is a bit-field or a subobject of zero size, the address of that object is the address of the first byte it occupies. Two objects with overlapping lifetimes that are not bit-fields may have the same address if
>
> — one is nested within the other,
>
> — at least one is a subobject of zero size and they are not of similar types ([conv.qual]),~~or~~
>
> — at least one is a synthesized object representation or element thereof, or
>
> — they are both potentially non-unique objects;
>
> otherwise, they have distinct addresses and occupy disjoint bytes of storage.

Modify §6.7.2 [intro.object]p14 as follows:

> Except during constant evaluation, an operation that begins the lifetime of an array of `unsigned char` or `std::byte` other than a synthesized object representation ([basic.types.general]) implicitly creates objects within the region of storage occupied by the array.

Edit §6.7.4 [basic.life]p1 as follows:

> […] The lifetime of an object of type `T` other than an element of a synthesized object representation ([basic.types.general]) begins when:
>
> — storage with the proper alignment and size for type `T` is obtained, and
>
> — if it is not a synthesized object representation, its initialization (if any) is complete (including vacuous initialization) ([dcl.init]),
>
> except […]. The lifetime of an object *o* of type `T` other than an element of a synthesized object representation ends when:
>
> — if `T` is a non-class type, the object is destroyed, or
>
> — if `T` is a class type, the destructor call starts, or
>
> — the storage which the object occupies is released, or is reused by an object that is ~~not~~neither nested within *o* ([intro.object]) nor nested within the object of which *o* is the object representation, if any ([basic.types.general]).
>
> When evaluating a *new-expression*, storage is considered reused after it is returned from the allocation function, but before the evaluation of the *new-initializer* ([expr.new]).
> [*Example 1*: […] — *end example*]
> A synthesized object representation is not considered to reuse the storage of any other object.

Insert a new paragraph after §6.7.4 [basic.life]p3 as follows:

> The lifetime of a reference begins when its initialization is complete. The lifetime of a reference ends as if it were a scalar object requiring storage.
>
> [*Note 1*: [class.base.init] describes the lifetime of base and member subobjects. —*end note*]
>
> For an object *o* of class type, the lifetimes of the elements of the synthesized object representation begin when the construction of *o* begins and end when the destruction of *o* completes. Otherwise, the lifetimes of the elements of the synthesized object representation (if any) are the lifetime of *o*.

Modify §6.8.1 [basic.types.general]p4 as follows, splitting it into two paragraphs, and add one paragraph after it:

> The *object representation* of a complete object type `T` is the sequence of *N* ~~`unsigned char` objects~~bytes taken up by a ~~non-bit-field~~ complete object of type `T`, where *N* equals `sizeof(T)`. The *value representation* of a type `T` is the set of bits in the object representation of `T` that participate in representing a value of type `T`.

Editor's note: The paragraph break should be inserted here.

For~~The object and value representation of~~ a ~~non-bit-field~~ complete object or a non-bit-field non-potentially-overlapping subobject ([intro.object]) of type *cv* `T`, the object and value representation are the bytes and bits, respectively, of the object corresponding to the object and value representation of its type; the object representation is considered to be an array of *N cv* `unsigned char` if the object occupies contiguous bytes of storage. The object representation of a bit-field object is the sequence of `N` bits taken up by the object, where `N` is the width of the bit-field (11.4.10). The value representation of a bit-field object is the set of bits in the object representation that participate in representing its value. Bits in the object representation of a type or object that are not part of the value representation are padding bits. For trivially copyable types, the value representation is a set of bits in the object representation that determines a value, which is one discrete element of an implementation-defined set of values.

Drafting note: The status quo does not specify even the number of bytes in the object representation of a subobject other than a bit-field. This is because of issues related to potentially-overlapping subobjects and was considered a pre-existing defect in the discussion of [CWG2519] (Jan 6, 2023 telecon). We leave object/value representations of potentially-overlapping subobjects unspecified here, while fixing non-potentially-overlapping subobjects.

For an object *o* with type *cv* `T` whose object representation is an array *A*:

- If *o* is a complete object of type "array of *cv* `unsigned char`", then *A* is *o*.
- Otherwise, if *o* is the sole element of a complete object *B* of type "array of 1 *cv* `unsigned char`", then *A* is *B*.
- Otherwise, *A* is said to be a *synthesized object representation*, and is distinct from any object that is not an object representation.
  - If *o* is of type *cv* `char`, *cv* `unsigned char`, or *cv* `std::byte`, then the value of the sole element of *A* is the value congruent ([basic.fundamental]) to the value of *o*.
  - Otherwise, if *o* is an array whose element type is *cv* `char`, *cv* `unsigned char`, or *cv* `std::byte`, then the value of each element of *A* is the value congruent to that of the corresponding element of *o*.
  - Otherwise, for each bit *b* in *o*, let *b*' be the corresponding bit of *A*. Let *p(b)* be the smallest subobject of *o* that contains *b* other than an inactive union member or subobject thereof. If *p(b)* is a union object or is not within its lifetime or has an indeterminate value, or if *b* is not part of the value representation of *p(b)*, then *b*' has indeterminate value. Otherwise, if *b* has an erroneous value, then *b*' has an erroneous value. Otherwise, *b*' has an unspecified value

that is neither indeterminate nor erroneous; such a bit retains its value until *p(b)* is subsequently modified.

[*Note:* Attempting to access an element of a synthesized object representation of a volatile object results in undefined behavior ([dcl.type.cv]). —*end note*]

[*Note:* An object representation is always a complete object. —*end note*]

Drafting note: It's not entirely clear why potentially-overlapping subobjects couldn't be allowed here; reading from the object representation of a potentially-overlapping subobject doesn't seem to pose the same problems as writing to it. But since potentially-overlapping subobjects were already carved out by [CWG43], even as the source of a copy, it seems wise to repeat the restriction here unless CWG is certain that the restriction is not needed.

Drafting note: Because an object representation is pointer-interconvertible with its first element (see below), this new rule would expand reachability if we allowed an array object that isn't a complete object to be its own object representation: the first element of that array would become pointer-interconvertible with whatever the array itself is pointer-interconvertible with. To prevent this, we must restrict the set of objects that are allowed to be their own object representation to complete objects only; you can already reach every byte of a complete `unsigned char` array from a pointer to its first element.

Modify §6.8.4 [basic.compound]p5 as follows:

Two objects *a* and *b* are *pointer-interconvertible* if:

- they are the same object, or
- one is a union object and the other is a non-static data member of that object ([class.union]), or
- one is a standard-layout class object and the other is the first non-static data member of that object or any base class subobject of that object ([class.mem]), or
- one is the object representation of the other, or the first element thereof, or
- there exists an object *c* such that *a* and *c* are pointer-interconvertible, and *c* and *b* are pointer-interconvertible.

If two objects are pointer-interconvertible, then they have the same address~~, and it is possible to obtain a pointer to one from a pointer to the other via a `reinterpret_cast` ([expr.reinterpret.cast])~~.

[*Note*: A `reinterpret_cast` ([expr.reinterpret.cast]) never converts a pointer to *a* to a pointer to *b* unless *a* and *b* are pointer-interconvertible. —*end note*]

[*Note*: An array object and its first element are not pointer-interconvertible, even though they have the same address, unless the array is an object representation. —*end note*]

Modify §7.2.1 [basic.lval]p11 as follows:

> An object of dynamic type $\texttt{T}_{\text{obj}}$ is *type-accessible* through a glvalue of type $\texttt{T}_{\text{ref}}$ if $\texttt{T}_{\text{ref}}$ is similar ([conv.qual]) to:
>
> — $\texttt{T}_{\text{obj}}$,
>
> — a type that is the signed or unsigned type corresponding to $\texttt{T}_{\text{obj}}$, or
>
> — a `char`~~`, unsigned char,`~~ or `std::byte` type, if the object is an element of an object representation ([basic.life.general]).
>
> If a program attempts to access ([defns.access]) the stored value of an object through a glvalue through which it is not type-accessible, the behavior is undefined. […]
> [*Note 11:* […]]
> [*Example 2:* An element of an object representation can be accessed through a glvalue of type `char`, `unsigned char`, `signed char`, `std::byte`, or a cv-qualified version of any of these types. —*end example*]

Drafting note: Because this paper doesn't address object representations of potentially-overlapping subobjects, we lack the wording to say what happens if a reference to such an object is cast to `char&`, `unsigned char&`, or `std::byte&`, and the resulting lvalue is accessed. Therefore, the wording above avoids giving the impression that such an access is well defined: if we claimed that it were well defined, we would have to specify the behavior. A similar issue arises when the object is discontiguous.

Modify §7.3.2 [conv.lval]p3.4, as amended by the proposed resolution of [CWG2901], as follows:

> — Otherwise, the object indicated by the glvalue is read ([defns.access]). Let *V* be the value contained in the object. If `T` is an integer type or *cv* `std::byte`, the prvalue result is the value of type `T` congruent ([basic.fundamental]) to *V*, and *V* otherwise. […]

Modify §7.6.1.9 [expr.static.cast]p13 as follows:

> […] Otherwise, if the original pointer value points to an object *a*, ~~and there is an object *b* of type similar to `T` that is pointer-interconvertible ([basic.compound]) with *a*, the result is a pointer to *b*. Otherwise, the pointer value is unchanged by the conversion.~~let *S* be the set of objects that are pointer-interconvertible with *a* and have type similar to `T`.
>
> — If *S* contains *a*, the result is a pointer to *a*.
>
> — Otherwise, the result is a member of *S* whose complete object is not a synthesized object representation if any such result would give the program defined behavior. If there are multiple possible results that

would give the program defined behavior, the result is an unspecified choice among them.

— Otherwise (i.e. when there are no such members of *S* that would give the program defined behavior), if *a*'s object representation is an array *A* and `T` is similar to the type of *A*, the result is a pointer to *A*.

— Otherwise, if *a*'s object representation is an array *A* and `T` is *cv* `unsigned char`, the result is a pointer to the first element of *a*'s object representation.

— Otherwise, if `T` is *cv* `std::byte`, *cv* `char`, or an array of one of these types, let `U` be the type obtained from `T` by replacing `std::byte` or `char` with `unsigned char`. If a `static_cast` of the operand to `U*` would yield a pointer to an object representation or element thereof, the result of the cast to `T*` is that pointer value.

— Otherwise, the result is a pointer to *a*.

Otherwise, if the original pointer value points past the end of an object *a*:

— If *a*'s object representation is an array *A* and `T` is similar to the type of *A*, the result is `&A + 1`.

— Otherwise, if *a*'s object representation is an array *A* and `T` is *cv* `unsigned char`, the result is a pointer past the last element of *A*.

— Otherwise, if `T` is *cv* `std::byte`, *cv* `char`, or an array of one of these types, let `U` be the type obtained from `T` by replacing `std::byte` or `char` with `unsigned char`. If a `static_cast` of the operand to `U*` would yield a pointer value defined by one of the above cases, the result of the cast to `T*` is that pointer value.

— Otherwise, the result is the value of the operand.

Drafting note: The case of multiple objects is a pre-existing defect: when a union has multiple members of type similar to `T`, a `static_cast` from `void*` to `T*` can yield a pointer to any of them. In cases that are allowed during constant evaluation, the above change ensures that there is no ambiguity about the result (*i.e.* the result always points to the original object). At runtime, the choice is unobservable except when some choices would result in lifetime-related UB, modifying a const object, or accessing a volatile object through a non-volatile glvalue.

Modify §7.6.6 [expr.add]p6 as follows:

For addition or subtraction, if the expressions `P` or `Q` have type "pointer to *cv* `T`"~~, where `T` and the array element type are not similar, the behavior is undefined.~~, one of the following shall hold:

— `T` is similar to the array element type, or

— `T` is similar to `char` or `std::byte` and the pointer value points to a (possibly-hypothetical) element of an object representation.

Otherwise, the behavior is undefined.

Modify §9.2.9.2 [dcl.type.cv]p5 as follows:

If an attempt is made to access an element *e* of a synthesized object representation ([basic.types.general]) and *e* overlaps the storage occupied by a volatile object (including a subobject), the behavior is undefined. Otherwise, the ~~The~~ semantics of an access through a volatile glvalue are implementation-defined. If an attempt is made to access an object defined with a volatile-qualified type through the use of a non-volatile glvalue, the behavior is undefined.

Modify §17.6.5 [ptr.launder]p2 as follows, relative to the CWG-approved resolution for LWG4130:

Preconditions: `p` represents the address *A* of a byte in memory. ~~An~~ There is an object *X* ~~whose type is similar ([conv.qual]) to `T` is~~ located at the address *A* such that

— *X*'s type is similar ([conv.qual]) to `T`,

— `T` is *cv* `std::byte` or *cv* `char`, and *X* is an element of an object representation ([basic.types.general]), or

— `T` is an array type whose element type is *cv* `std::byte` or *cv* `char`, and *X* is an object representation

~~, and~~and such that *X* is either within its lifetime ([basic.life]) or is an array element subobject whose containing array object is within its lifetime. All bytes of storage that would be reachable through ([basic.compound]) the result are reachable through `p`.

Modify §17.6.5 [ptr.launder]p3 as follows:

Returns: A value of type `T*` that points to the object *X* that would give the program defined behavior, or to an unspecified choice among them if more than one such object exists. If no such object exists, the behavior is undefined.

§ 9 Document history

— **R0**, 2019-07-30: Initial version.

— **R1**, 2019-09-28: Allowed pointer arithmetic on expressions of type `unsigned char*`, `char*` and `std::byte*` when pointing to objects of different type. Removed exclusion of the object representation of objects of zero size from appearing in the object representation of their containing object. Added multi-dimensional arrays of

contiguous-layout types to the definition of contiguous-layout types. Slight change to the behavior of `std::launder` for when there are multiple viable objects.

— **R2**, 2019-11-20: Removed contiguous-layout types from wording, this should be tackled by [P1945R0].

— **R3**, 2022-02-15: Moved wording for casts to the rules of pointer-interconvertibility. Changed the wording for `std::launder` to bind to the best candidate object.

— **R4**, 2022-03-16: Changed the wording to fix ambiguous usage of *N* in object representations specification.

— **R5**, 2022-06-16: Reduced scope of paper to only reading object representations, not writing. Completely rewrote rationale. Added wording to prevent implicit object creation within object representations. Added cross-reference to types with contiguous storage ([intro.object]) in the wording. Fixed inconsistency in the wording by defining that only `unsigned char` is its own object representation, not `char` or `std::byte`. Removed erroneous wording regarding memory locations. Added list of known issues.

— **R6**, 2024-09-26: Converted to HTML (generated from Markdown). Rebased wording on N4988. Removed unnecessary speculation about the behaviour of modifying object representations. Made elements of a subobject object representation distinct objects from the elements of the containing object's object representation and adjusted wording accordingly. Clarified which bits of object representations are indeterminate or erroneous. Removed some ambiguity over the result of `reinterpret_cast` and added wording for the case of past-the-end pointers and casts to `std::byte*`. Made the object representation of a non-contiguous object no longer consist of *cv* `unsigned char` objects. Defined object representation of (some) subobjects. Fixed a wording bug for pointer arithmetic.

— **R7**, 2025-01-10: Fixes after CWG review:

  — Accessing object representation is no longer permitted for volatile objects.

  — `std::launder` now gives an unspecified choice in the case of multiple objects of the same type, and allows the case where `T` is of type *cv* `std::byte` or array thereof: you can get a pointer to the object representation or array thereof.

  — The lifetimes of elements of synthesized object representations are now considered to cover the entire period of construction and destruction. We also clarify that a synthesized object representation doesn't have to be initialized (so its lifetime starts as soon as suitable storage is obtained) and that its storage isn't considered reused by objects that are nested within the actual object.

  — The description of *p(b)* now excludes inactive union members and their subobjects, so you get an active union member if possible, otherwise the bit is treated as if it were a padding bit.

— Unspecified object representation bit values are now explicitly specified to be stable (but the value is still unspecified) and to not be indeterminate nor erroneous.

— Added “congruent to” for determining the value of an element of an object representation of a `char` or array of `char`.

— Subobject object representations are no longer considered to be nested within enclosing objects’ object representations. Instead, we carve out explicit exceptions to allow object representations to share storage and to not reuse storage of another object. We also clarify that synthesized object representations never provide storage.

— The wording has been improved for object representations of non-potentially-overlapping subobjects.

— Added explanation for why only *complete* arrays of `unsigned char` are allowed to be their own object representations.

— The strict aliasing rule has been modified to reflect the fact that access to object representations is now access to `unsigned char` objects, not to the original object.

— The wording for obtaining a pointer to an element of an object representation by casting to `char*` has been restored; it appears to have been removed accidentally in R3.

— Removed the “would be well-formed” condition from [expr.static.cast]p13: the cases that cast away constness are checked earlier in the paragraph.

§ 10 Acknowledgements

Many thanks to Jens Maurer and Hubert Tong for their help with the wording. Thanks to Janet Cobb, John Iacino, Marcell Kiss, Killian Long, Theodoric Stier, and everyone who participated on the std-proposals mailing list and Core reflector for their countless reviews and suggestions for earlier revisions of this paper. Thanks to Professor Ben Woodard for his grammatical review of an earlier revision of this paper.

§ 11 References

[CWG1314] Nikolay Ivchenkov. 2011-05-06. Pointer arithmetic within standard-layout objects.
https://wg21.link/cwg1314

[CWG2519] Jiang An. 2022-01-20. Object representation of a bit-field.
https://wg21.link/cwg2519

[CWG2901] Jan Schultke. 2024-06-14. Unclear semantics for near-match aliased access.
https://wg21.link/cwg2901

[CWG43] Nathan Myers. 1998-09-15. Copying base classes (PODs) using memcpy.
https://wg21.link/cwg43
[N5001] Thomas Köppe. 2024-12-17. Working Draft, Programming Languages — C++.
https://wg21.link/n5001
[P0137R1] Richard Smith. 2016-06-23. Core Issue 1776: Replacement of class objects containing reference members.
https://wg21.link/p0137r1
[P0593R6] Richard Smith, Ville Voutilainen. 2020-02-14. Implicit creation of objects for low-level object manipulation.
https://wg21.link/p0593r6
[P1945R0] Krystian Stasiowski. 2019-10-28. Making More Objects Contiguous.
https://wg21.link/p1945r0

---

1. Since C++20, one can also use `std::bit_cast` to copy the bytes into a struct that contains an array of `unsigned char`, assuming that the struct does not have any padding.↩

2. All citations to the Standard are to working draft N5001 unless otherwise specified.↩

3. These reachability-based restrictions limit compatibility between C and C++, in particular when it comes to C code that uses `offsetof` to implement intrusive data structures. A separate paper, P3407R0, proposes to remove these restrictions. Additional specification difficulties are raised by such a direction, which will not be discussed here.↩